

What v2 needs to be

You've built a working agentic contract reviewer. You want to sell it. Those are two different products, and the gap between them is not mostly about being smarter — it's about being *trustworthy, provable, and installable*.

Written from a full read of the v1 codebase · 18 August 2026 · Target: a product to sell · Scope: open-ended

DO THIS FIRST — 2 MINUTES

Your live OpenRouter key sits in plaintext in `config.yaml`, and that file was read into this session. **Rotate the key now** at openrouter.ai/keys. Separately, treat "secrets live in a YAML file next to the code" as a v2 blocker in its own right — it's the first thing a law firm's IT reviewer will find, and it fails on sight.

01 THE HONEST STARTING POSITION

You have one genuinely rare asset and one prototype wrapped around it. Be clear about which is which, because only one of them is worth defending.

What's rare

- **The skill file is the product.** 4,500 words of actual senior-practitioner judgement — minimum effective intervention, existing drafting as evidence, the duplicated-protection rule, the consequence test, the indemnity/cap/gross-up reasoning patterns, the internal-advice vs bubble-comment split. Nobody can prompt their way to §8.2 or §3.3. This is the moat.
- **The architecture is right, and most competitors get it wrong.** A supervisor that reads through tools rather than swallowing the document; a deterministic layer

that cannot hallucinate; verbatim quotation checked against the text before the Insert button is even offered. That last one — `anchor_verified` — is the single best trust feature in the codebase and you're currently under-selling it.

- **Indian-law fluency is baked in and nobody else has it.** Your number parser handles lakh and crore. Your eval document is an SSA with promoter joint-and-several liability and a shareholding gross-up. Spellbook and Legora do not think in these terms.

What's a prototype

Everything else. Single-user global config, no persistence of any kind, no auth on the API, no audit log, no cost visibility, no licensing, PowerShell sideload, a self-signed certificate, and one manually captured eval run. None of that is a criticism of v1 — it's exactly the right amount of scaffolding for proving the idea. It's just all load-bearing the moment someone else pays for it.

02 FIVE THINGS THAT BLOCK THE SALE

These aren't features. They're the questions a law firm's procurement and IT functions will ask, in roughly this order, and today you lose on all five.

1. The confidentiality story doesn't survive diligence **BLOCKER**

This is the one that kills deals, so it goes first. Three separate problems compound:

- The API key is plaintext on disk.
- OpenRouter fans out to whichever provider serves the slug — so "where did my client's SPA go" has no clean answer, and the answer changes run to run.
- **Worst of the three:** `Router.resolve()` falls through to `pool[0]` — the first model in whatever order OpenRouter happened to return — when no preference prefix matches. Today the default prefix list is broad enough that this rarely fires, but it fires immediately under `require_only_free: true` or any narrowed `prefer` list. That path silently routes a confidential agreement to an arbitrary model. (Relatedly: your captured eval ran the supervisor on `moonshotai/kimi-k3`

— whether pinned deliberately or resolved, the point is that nothing in the system records or warns which provider saw the document.)

v2: Fail closed, never fall through to an arbitrary model. Ship an explicit allowlist. Then add the enterprise answer: **bring-your-own model endpoint** — Azure OpenAI or Bedrock inside the customer's own tenant, so the document never leaves their cloud boundary and you never touch it. Move the key to the OS credential store. Surface, per run, exactly which model and provider handled the document, and log it.

Why this is also your wedge: the incumbents are cloud SaaS. "Your document never leaves your tenant" is a positioning advantage you already half-have — you just haven't finished it or said it out loud.

2. It can't be installed by a normal person **BLOCKER**

Today: run `setup.ps1`, trust a self-signed cert, sideload a manifest, keep a Python server alive. That's a demo, not a distribution channel.

v2: Microsoft Marketplace / AppSource listing, plus centralized deployment through the M365 admin center so a firm's IT can push it to 200 lawyers at once. Budget real time for this — Partner Center validation is a process, not a form, and it will force decisions about hosting and privacy policy that you should make deliberately anyway.

3. There is no audit trail **BLOCKER**

Nothing is written to disk. The event stream already exists — `parsed`, `tool`, `issue`, `done` — and it's thrown away when the pane closes. Your `_eval_mode_a.jsonl` is a hand-capture of exactly the thing you should be persisting automatically.

v2: Every run persisted: document fingerprint, mandate, mode, model, tool calls, issues, what the user *accepted*, *rejected*, or *edited*, tokens, cost, elapsed. This is cheap to build and it pays three times over — it's the compliance artifact, the debugging tool, and (see §04) the raw material for the memory that makes the product compound.

4. Nothing stops a second user from breaking it **BLOCKER**

`cfg` is a module-level global that `/api/settings` mutates in place; one `Router` is shared; there is no tenancy, no accounts, no auth on the local API at all. Any process on the machine can POST a document to `127.0.0.1:8787`.

v2: This is the fork in the road. **Local-first** (the add-in talks to a per-user local service, licensing checked online, documents never centralised) or **hosted** (easier billing and updates, harder legal sale). See §08 — it's the first decision to make, because it determines almost everything downstream.

5. There's no way to charge for it BLOCKER

No accounts, no entitlement check, no metering. And with BYO-OpenRouter-key you've handed the customer your variable cost — which sounds generous but actually makes you unsellable to firms, because no partner wants to manage an API key or explain a usage bill to finance.

v2: Decide whether you resell inference (simpler for the buyer, margin risk for you, and the reason every competitor hides pricing) or stay BYO-key for enterprise and resell only on a self-serve tier. Either way you need licence keys, seat counts, and a metering path — and you can't build metering without §05's cost instrumentation.

03 THREE CAPABILITY JUMPS WORTH PAYING FOR

Fixing the blockers makes it sellable. These make it *worth buying* — and each one is a real gap in v1, not a nice-to-have.

Jump 1 — Let it see the whole document CAPABILITY

`readParagraphs()` pulls `document.body.paragraphs` — text plus list numbering. That's it. Which means:

- **It cannot see Word comments.** Mode N explicitly instructs the agent to check "unresolved comments." It structurally cannot. That mode is quietly lying to the user.
- **It cannot see tracked changes.** Mode F is "counterparty markup review" — the agent sees only the current text state, never what the counterparty actually

changed. The whole mode is running blind on its central input.

- **Tables lose their shape.** Cell text arrives as loose paragraphs with no row or column relationship. Warranty schedules, CP lists, cap tables and disclosure schedules — the places where pre-signing defects actually hide — are exactly what this flattens.
- **Headers, footers and footnotes are excluded entirely** by `body.paragraphs`. Execution blocks, page-level defined terms, footnoted carve-outs: invisible.

v2: Ingest comments, revisions, tables (as structure), headers/footers and footnotes. Then add the tools the supervisor needs to use them:

`get_tracked_changes`, `get_comments`, `read_table`. Modes F and N go from half-blind to genuinely best-in-class, and F is the mode lawyers actually run every day.

Jump 2 — The unit of work is the matter, not the document CAPABILITY

`ReviewRequest` accepts one `paragraphs` list. There is nowhere to put a second document. So **Mode L — comparison of versions — has a mode brief but no mechanism. It cannot work as built.**

The deeper point: transactions are document *sets*. SSA plus SHA plus disclosure letter plus side letter plus board resolutions. A large share of the defects that matter are cross-document — SHA reserved matters contradicting SSA covenants, disclosure letter paragraph numbers not mapping to warranty numbers, a side letter quietly overriding a cap. No competitor does this well, and your overlap check (§3.5 of the skill) and deal map already generalise to it almost for free.

v2: A "matter" object holding N documents. Same tool surface, extended with a document parameter. New modes: cross-document consistency, disclosure-letter mapping, version comparison that actually has two versions to compare. **This is the most impressive demo you can build**, and it's the one an incumbent can't quickly copy.

Jump 3 — Memory that compounds CAPABILITY MOAT

Every run starts cold. `Toolbox` holds issues in a Python list that dies with the request; the only thing that persists anywhere is the mandate, in `localStorage`. Meanwhile §11 of your skill is titled "Evidence Hierarchy for Personalised Preferences" and §12

lists "frequently preferred negotiating positions" — both written as if the agent learns. It doesn't. Every correction you make in Word is a training signal you throw in the bin.

v2, in three layers:

- **House positions.** Capture accept / reject / edit per issue. When you rewrite the agent's proposed wording, store the delta. Feed the resulting positions file into the system prompt. The agent stops re-proposing things you've declined three times.
- **Precedent bank.** Index the firm's executed agreements. When the agent proposes a cap, it retrieves *your own* prior language for the same concept. \$12 of your skill warns against pretending a preferred position is market — a precedent bank is the honest answer to that, and it's exactly the ground DraftWise has been taking.
- **Matter memory.** Round two of a negotiation should know what was conceded in round one.

This is what converts a tool into an asset that gets more valuable per month of use — and it's the thing that makes churn low, which is the thing that makes the business work.

04 YOU CAN'T SELL WHAT YOU CAN'T PROVE

One captured Mode A run and an offline test suite with a stubbed router. That's fine for a personal tool. For a product it's the largest single gap on this page.

There's a specific, underappreciated danger in your design. Model resolution is *dynamic*: `resolve()` sorts by `created` and takes the newest release in a family. You wrote that deliberately, and it's elegant — the tool follows the frontier without an update. But it also means **your product's behaviour changes silently whenever a provider ships**. A new release can regress recall on cross-references, or start over-flagging, and you would have no signal until a customer told you. For a personal tool that's a shrug. For a product sold to lawyers it's an unacceptable exposure.

Build the golden set

- **15–25 agreements with a labelled defect list.** Real Indian SSAs, SHAs, MSAs, SaaS agreements — with known planted and naturally-occurring defects. This is slow, boring, and the highest-leverage work on this page.
- **Score every run:** recall against known defects, false-positive rate, `anchor_verified` pass rate, severity calibration, cost, wall time.
- **Gate on it.** No model change ships without a run. Pin models for customers; promote a new one only after it clears the bar.

Two side benefits that matter more than they look. First, this is how you answer "is Opus worth 6× Sonnet for the supervisor role?" — a question you currently cannot answer, and which is your entire gross margin. Second, **the eval results are a sales asset.** "We measure recall on a benchmark of 22 Indian venture financing documents" is a sentence no competitor's marketing page can match, and it's exactly what a sceptical partner needs to hear.

05 THE ECONOMICS YOU CURRENTLY CAN'T SEE

OpenRouter returns a `usage` block on every call. You discard it. So a 40-step supervisor loop with up to 25 delegations has a cost that is both unbounded and invisible.

WHAT'S HAPPENING	CONSEQUENCE	V2 FIX
Usage discarded	No cost per run, per role, or per document. Can't price, can't meter, can't optimise.	Capture and surface it. Prerequisite for billing.
Full history resent each step	The 4,500-word skill plus operating rules plus mode brief go up on every one of 40 calls, with the transcript growing underneath.	Prompt caching on the system block. Largest single cost lever available.
Whole skill always loaded	Running Mode B ships thirteen irrelevant mode specifications. Cost <i>and</i> attention dilution.	Sectioned loading: always-on core (\$3, \$5, \$10, \$13) plus the active mode only.

WHAT'S HAPPENING	CONSEQUENCE	V2 FIX
No budget ceiling	A long SHA can burn an unpredictable amount with no cap.	Hard cost ceiling per run, with a graceful "here's what I found before I stopped."
No latency instrumentation	You don't know whether runs feel slow because of the model, the loop, or the delegations.	Per-step timing in the run log.

There are also two silent truncation limits worth knowing about before a customer finds them: `outline()` caps at 400 items and `search()` at 40 hits. On a 200-page SHA with schedules, both will bite — and the agent won't be told it's seeing a partial view.

06 WHERE YOU'D SIT IN THE MARKET

The category is real, funded, and crowded at the top. It is not crowded where you are.

TOOL	REPORTED PRICE	SHAPE
Harvey	~\$1,000–2,000+ /user/mo	Enterprise legal platform, big-law
Legora	~\$300–800 /user/mo	Enterprise, collaborative review
LegalOn	~\$550 /mo (individual, annual)	Playbook-driven review
CoCounsel	~\$225–400+ /user/mo	Bundled with Westlaw
Spellbook	~\$99–350 /user/mo	Word add-in — your closest analogue

Prices are reported/estimated — every one of these vendors keeps list pricing private, which is itself a signal about how variable inference cost is.

Your wedge, stated plainly

Don't compete with Harvey. Compete where the incumbents are structurally weak:

- **Indian and cross-border venture/PE documentation.** SSAs, SHAs, promoter liability, tag/drag, reserved matters, lakh/crore. The global tools are trained on US/UK market practice and quietly wrong on Indian deal structure. You already parse crore. Nobody else does.
- **Verified, not plausible.** Deterministic checks plus quotation verification plus a published benchmark. Every competitor asks lawyers to trust a black box. You can show your working — and lawyers, of all buyers, will pay for that.
- **Runs in the customer's tenant.** Finish the local-first story and it becomes the reason a paranoid firm picks you over a US cloud vendor.
- **The boutique/solo corporate practice** that can't justify \$800/seat but does the same work. Underserved and reachable without an enterprise sales team.

07 CHEAP DETERMINISTIC WINS

The mechanical layer is your trust engine — instant, free, unhallucinable. It has six checks. It should have twenty. This is the best value-per-hour work in the whole project.

Structural CHEAP

- **Orphan schedules** — you check references→targets but never targets→references. A Schedule 4 nobody refers to is a classic defect you currently miss.
- **Empty schedules** — a heading with nothing under it. Your own eval found three by model reasoning; this should be free.
- **Chapeau-less parents** — a Clause 5 with 5.1 and 5.2 but no lead-in.
- **Signature block vs party list** mismatch.

Semantic-adjacent CHEAP

- **Forward-defined terms** — a term used operatively before it's defined.
- **Date logic** — Closing before Execution, cure period exceeding the term, survival period shorter than the limitation period.
- **Party-name drift** — "the Company" vs "Company" vs the registered name.
- **Cross-reference plausibility** — Clause 3.2 exists, but its heading is "Notices" and it's cited as the payment mechanic. Heading-mismatch heuristic catches a

surprising number of copy-paste errors.

- **Currency and unit consistency** across the document.

08 A SEQUENCE THAT WORKS

Ordered so that each phase makes the next one possible. Resist reordering — the temptation will be to build Jump 2 first because it demos well, but it's much harder without the persistence layer underneath it.

PHASE 0

Days

Stop the bleeding

- Rotate the key. Move secrets to the OS credential store.
- Make `Router.resolve()` fail closed. Ship an explicit model allowlist.
- Capture `usage` and per-step timing. Log every run to disk.
- Add prompt caching on the system block.

PHASE 1

Weeks 1–4

Make the claims provable

- Build the golden set — 15–25 labelled agreements. Start now; it's the long pole.
- Scoring harness: recall, false positives, anchor-verification rate, cost, latency.
- Six to eight new deterministic checks from §07. Each one is a scored, provable gain.
- Persistence layer: matters, documents, runs, issues, and *user disposition per issue*. Everything downstream depends on this schema — design it once, carefully.

PHASE 2

Weeks 4–10

The capability jumps

- Full ingestion: comments, tracked changes, tables, headers/footers/footnotes — plus the tools to use them. Modes F and N become real.

- Multi-document matters. Mode L finally works; cross-document consistency becomes a new mode and your best demo.
- House positions learned from accept/reject/edit. Sectioned skill loading.
- Exportable deliverables — issues memo as Word, Mode M email as a real draft, not just prose in `finish()`.

PHASE 3

Weeks 10+

Make it a business

- Accounts, licensing, entitlement, metering.
- Bring-your-own tenant model endpoints (Azure OpenAI / Bedrock). This is the enterprise unlock.
- AppSource listing plus M365 centralized deployment.
- Security posture: audit log surfaced to admins, data-handling documentation, and — when the first real firm asks — start the SOC 2 Type II clock. Assume it's asked for.
- Precedent bank on top of the persistence layer built in Phase 1.

09 THE DECISIONS ONLY YOU CAN MAKE

Four forks. My lean is marked, but these are judgement calls about your market, not your code — and getting them wrong costs months.

Local-first, or hosted?

Local-first is a genuine differentiator in legal and you're already 80% there. It's also harder to update, harder to support, harder to meter, and rules out server-side precedent search across a firm.

Lean: local-first execution with a hosted control plane — licensing, updates and telemetry go to your server; documents and model calls never do. Keeps the privacy story intact while making the business operable.

Who is the buyer — the boutique, or the firm?

A boutique corporate practice buys on your judgement and buys fast. A large firm buys on security posture, integrations and procurement, and takes nine months. Both are viable. Trying to build for both at once is how this stalls.

Lean: boutique and solo corporate practices in India first. They feel the pain most acutely, they can't afford the incumbents, they'll tell you the truth, and their feedback loop is measured in days.

Do you resell inference, or stay bring-your-own-key?

BYO-key keeps your gross margin clean and your liability low, but it's an adoption tax most lawyers won't pay and it makes usage-based pricing impossible.

Lean: both, deliberately — you resell on the self-serve tier (simple flat price, cost ceiling per run, which you can only enforce once \$05 is built), BYO-tenant on enterprise. Same product, two commercial shapes.

Does the skill file stay one file, or become a product surface?

Right now the skill is the agent, and it's yours — your positions, your risk appetite. A firm buying this will want its own playbook. That's a real feature (LegalOn's whole pitch) and also a way to give away your edge.

Lean: split it. A locked core — the reasoning methodology in \$3, \$5, \$7, \$8, \$13 — that customers get but can't edit, plus a customer playbook layer for house positions and preferred fallbacks. Sell the methodology, let them own the positions.

10 IF YOU ONLY DO THREE THINGS

1. **Build the golden set and the scoring harness.** Everything else is unfalsifiable without it, and you're currently shipping model changes blind.
2. **Build the persistence layer — including what the user did with each issue.** It unlocks the audit trail, the learned positions, the precedent bank and the metering. Four requirements, one schema. Design it once and design it well.
3. **Fix the routing fallback and finish the confidentiality story.** It's a small code change and it's the difference between a tool a lawyer would try and a product a firm would approve.

Sources for market and distribution context: [Legal AI tool pricing \(Spellbook\)](#) · [Legora vs Harvey](#) · [DraftWise — precedent intelligence](#) · [Legal AI tools reviewed, 2026](#) · [Publishing Office Add-ins to Microsoft Marketplace](#) · [Data security in legal AI](#)

All statements about the current codebase were verified against the files in `D:\AI Agent\agreement-agent` on 18 August 2026.